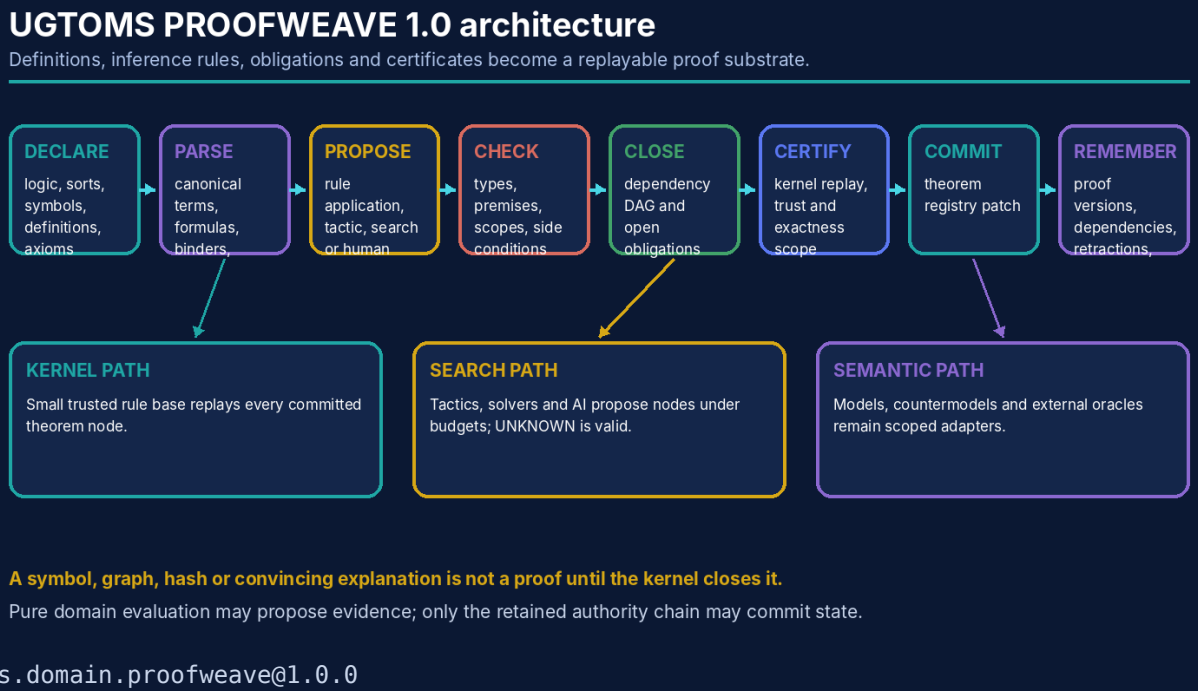
UNIVERSAL GEOMETRIC TOPOLOGICAL
OPEN MODULAR SUBSTRATE

UGTOMS PROOFWEAVE 1.0

Content-Addressed Proof Graph, Kernel and Certificate Cartridge



A symbol, graph, hash or convincing explanation is not a proof until the kernel closes it.

Pure domain evaluation may propose evidence; only the retained authority chain may commit state.

`ugtoms.domain.proofweave@1.0.0`

Component	PROOFWEAVE
Canonical identity	ugtoms.domain.proofweave@1.0.0
Component type	Formal domain cartridge
Version / date	1.0.0 / 30 August 2026
Document type	Open modular cartridge specification and implementation-ready profile
Status	Formal proof cartridge; specification self-checked; universal soundness, consistency and automated theorem proving not claimed

ATTRIBUTION AND EVIDENCE BOUNDARY

Prepared for Tom Klootwijk. Requester attribution is recorded as supplied and is not independently verified. This is a technical design artifact, not legal proof, scientific validation, regulatory approval or production safety certification.

Contents and Reading Rule

READING RULE

The UGTOMS source line supplies the reusable authority, typing, order, evidence and packing discipline. The specialist domain model in this cartridge is an engineering-derived extension. Where the source corpus is silent, this report says so instead of presenting general domain knowledge as source-derived.

Executive Decision and Release Scope

FORMAL DECISION

GO for PROOFWEAVE as the proof-carrying cartridge. A theorem becomes authoritative only when a small declared kernel can replay a closed content-addressed proof graph under exact theory, scope and trust metadata. Human, tactic, search, SAT/SMT and AI systems remain proposal or certificate adapters.

PROOFWEAVE strengthens the entire UGTOMS ecosystem because it provides one place to express what a certificate actually proves, which definitions and assumptions it depends on, and which obligations remain open. It carries proof objects; it does not claim to settle every theorem or validate every external premise.

- Syntactic derivation, semantic entailment, numerical evidence and empirical observation remain different typed relations.
- A proof search that exhausts its budget returns UNKNOWN rather than inventing a proof or refutation.
- Chess and Go demonstrate the intended evidence discipline: exact results are promoted only when finite obligations close and certificates replay [S12-S13].

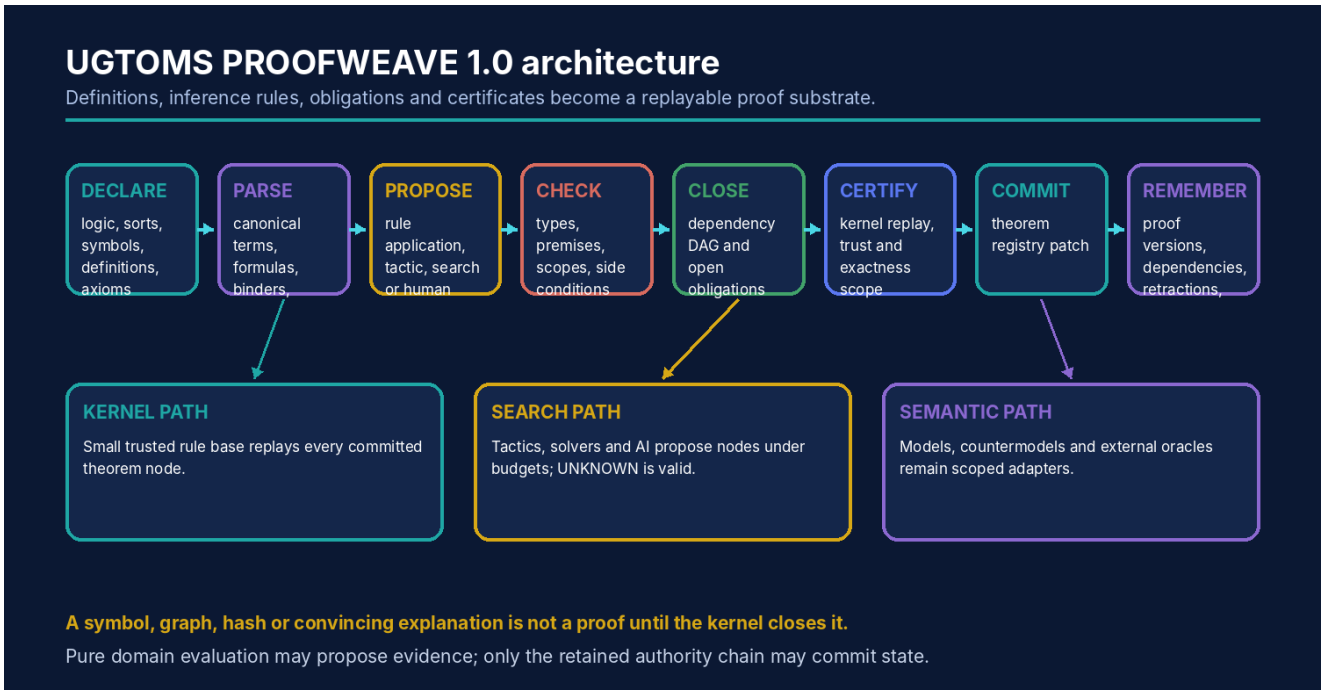


Figure 1. PROOFWEAVE places proof search upstream of a small replayable kernel and theorem commit.

Source Basis and Evidence Discipline

The cartridge inherits the following UGTOMS commitments:

- Typed definitions and operators live inside the substrate, with stable IDs, dependencies and content hashes [S1-S3].
- Pure evaluation is upstream of support, compatibility, guard, verified proposal, deterministic commit and lineage [S1, S2, S4].
- Coordinates, pictures, sounds, meshes and packed codes are not identity or authority by themselves [S4-S8].
- Uncertainty, exactness, capability and error margins are explicit and may only be strengthened by evidence [S1, S2, S8].
- Component-scoped identities prevent a specialist domain pack from masquerading as a universal replacement of every other release [S10].

ENGINEERING-DERIVED DOMAIN CONTENT

The theory/syntax records, inference schemas, kernel, proof-DAG format, semantic/oracle interfaces and example calculus are engineering-derived. The source corpus supplies content-addressed definitions, typed operators, exactness/order discipline and examples of bounded proof certificates; it does not supply a universal logic or proof assistant.

ID	Registered source	Use in this cartridge	Pages
S1	UGTS__FOUNDATION__UNICODE_v5.pdf	Foundation 5.0 OperatorCells, literal/glyph/semantic separation, converse relations, exactness/capability and packed execution boundaries.	pp. 1-21
S2	UGTS_KC_4_2_General_Operator_Order_Addendum(1).pdf	Typed OperatorSpec, dependency and phase order, deterministic reduction, certificates, trace queries and open extension contracts.	pp. 2-32
S3	UGTS_KC_3_6_Tom_Klootwijk.pdf	Definitions as first-class content-addressed records, resolvable references, explicit operation order and deterministic trace.	pp. 1-15
S4	Unified_Geometric_Topological_Substrate_GPU_Native_Addendum.pdf	Canonical support → compatibility → guard → event → transition → lineage authority and narrow packed-state semantics.	pp. 2-15
S5	UGTS_KC_2_0_Tom_Klootwijk_Expanded_Substrate.pdf	Typed state, patterns, topology, bounded dynamics, hybrid events and numerical/error contracts.	pp. 3-15
S6	UGTS_KC_Two_Hands_3_0_Interactive_Graphics_Runtime.pdf	Stable identity, proposal verification, deterministic commit, checkpoints, rollback and replay.	pp. 3-16
S7	UGTS_KC_3_9_KC_Elizabeth_Vector_Game_Runtime.pdf	Versioned game projects, fixed-step runtime, snapshots/hashes, CLI and self-contained HTML delivery.	pp. 2-16
S8	UGTS_KC_4_0_Spatial_Evidence_Ledger(1).pdf	Observation versus authority, uncertainty, provenance, atomic patches, checkpoints and evidence-bearing lineage.	pp. 3-14
S9	UGTS_KC_3_6_2_SCLP_Tom_Klootwijk.pdf	Finite packing, interval guards, winding, bounded grammars and explicit topology/rotation distinctions.	pp. 1-15
S10	UGTS_Versioning_Charter_Phase_1_Foundation_Chronology(1).pdf	Component-scoped identities and explicit release graphs instead of one misleading global version ladder.	pp. 1-6
S11	UGTS_KC_Definition_Breaker_DB_0_1_Concept(1).pdf	Rules as typed physical objects, candidate preview, deterministic verification, commit, replay and reason-coded rejection.	pp. 2-24
S12	UGTS_KC_CHESS_1_0_Proof_Carrying_Solver(1).pdf	Exact rule authority, bounded proof certificates, heuristic separation and independently verifiable replay.	pp. 2-20
S13	UGTS_KC_4_2_GO_Solver_Report(1).pdf	History-correct exact state, finite minimax certificates, search/non-proof separation and self-contained delivery.	pp. 1-15

Open Modular Cartridge ABI

The cartridge is a concrete instance of the shared domain object D_PROOFWEAVE:

```
D = (T, O, R, U, I, Q, E, G, Delta, L, Cert, K, Pi)
```

Field	Contract
T	Entity, value and state types
O	Typed operators and transformations
R	Relations, graphs, topology and converse pairs
U	Units, dimensions, coordinate and profile conventions
I	Invariants, conservation laws and constraints
Q	Dependency, precedence, phase and reduction order
E	Observations, evidence, uncertainty and provenance
G	Guards, thresholds and admission predicates
Delta	Proposed and committed state patches
L	Lineage, replay, history and migration
Cert	Exactness, residual, interval, scope and status certificates
K	Cold atlas, hot codebook and packed representation
Pi	Optional visual, audio, physical or device projection

A conforming cartridge contains a manifest, type registry, operator registry, relation atlas, unit/profile system, dependency DAG, guard library, transition definitions, claims ledger, examples and validation fixtures. External mature libraries may sit behind adapters when they are safer or more accurate than a fresh reimplementation.

General Operator Record and Order

```
OperatorSpec = (id, version, syntax, arity, domains, codomain, units, shape,
                laws, exactness, effects, capabilities, dependencies,
                order_contract, invariants, provenance, content_hash)
```

The record follows the General Operator and Order layer [S2]. A symbol or glyph is not allowed to smuggle in undeclared type, unit, order, inverse or authority semantics.

EXAMPLE RECORD

proofweave.rule.apply consumes an InferenceRule, exact premise nodes, typed substitutions and current Context; it returns a CandidateProofNode plus a side-condition trace. The candidate enters the theorem registry only after the kernel replays it and every dependency is already committed or included in the same closed certificate.

Normative phase schedule

Phase	Meaning
1-3	parse, normalize and resolve content-addressed references
4-6	type, shape and unit validation; dependency closure
7-9	construct candidates; apply declared composition and reduction order
10-12	evaluate exact or bounded results; issue capability and residual certificate

Phase	Meaning
13	support, compatibility and guard classification
14	verified proposal and deterministic conflict resolution
15	atomic commit, lineage, replay and optional projection

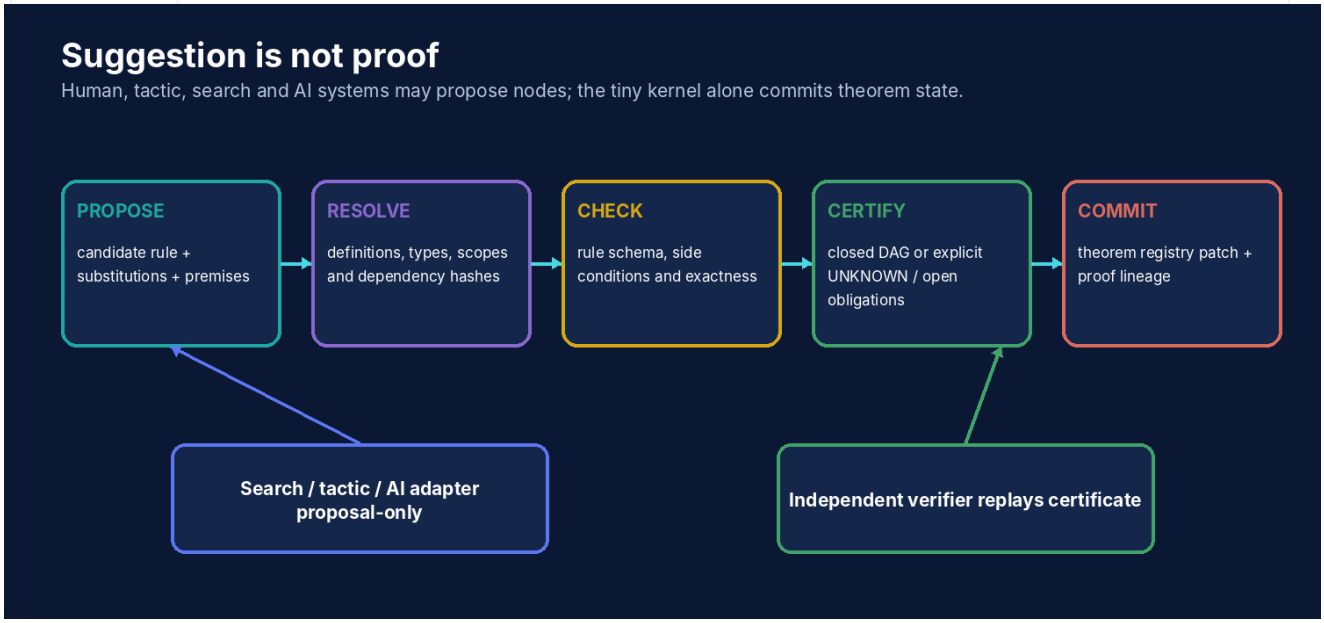


Figure 2. Suggestions from any source remain proposal-only until kernel verification and atomic theorem commit.

Formal Proof State and Trust Boundary

```
q_PFW = (P, Sigma, Gamma, G, N, O, K, C, L, U)

P theory/logic profile      Sigma symbols, sorts, definitions, axioms
Gamma local contexts        G goals and constraints
N proof DAG nodes           O open obligations
K kernel/trust configuration C certificates and theorem registry
L proof/version lineage     U status, residual and UNKNOWN reasons
```

Object	Authority role	Boundary
TheoryProfile	Logic, sorts, equality, definitions, axioms, rule set and trust base.	A theorem cannot silently migrate between theories.
Context	Local declarations and assumptions with exact scope/discharge state.	Visual nesting or indentation is not scope authority.
ProofNode	Conclusion, rule, premises, substitutions, side conditions, scope and hash.	Natural-language explanation is a projection, not the node itself.
ProofCertificate	Closed root, complete node set, dependencies, theory/trust profile and integrity.	A hash alone is not proof.
TheoremRegistry	Committed theorem statements and proof roots with versions and lineage.	Search cache or candidate goals are not theorems.

TRUST-BASE RULE

Every certificate names trusted axioms, definitions, kernel rules, admitted facts and external certificates. Soundness is always relative to that declared base; no local proof silently proves the base consistent or empirically true.

Theories, Syntax and Content-Addressed Definitions

```
TheoryProfile = (logic_id, sort_system, symbol_table, equality_profile,
                 definition_policy, axiom_set, inference_rules,
                 normalization, semantic_profiles, trust_base, hash)
```

Layer	Canonical object	Key distinction
Literal	Unicode/ASCII source spelling and source locations	spelling is not semantic identity alone
Syntax	term/formula AST with binder IDs and types	AST is distinct from rendered glyph geometry
Definition	content-addressed mapping plus transparency/unfolding policy	definition equality differs from asserted theorem equality
Proposition	typed formula under a theory/context	statement does not include its proof automatically
Theorem	proposition plus committed proof root and trust metadata	same proposition may have multiple proof lineages

Binder and substitution discipline

- Free and bound variables carry stable binder identity, not only printed names.
- Substitution is typed and capture-avoiding; alpha normalization may canonicalize names under a declared profile.
- Definition unfolding, rewriting and normalization record exact strategy, fuel and residual/open status.

REFERENCE CLOSURE

Missing symbols, duplicate IDs, bad content hashes, unresolved definitions and undeclared dependency cycles are validation failures before proof checking begins [S3].

Inference Rules, Substitution and Scope

```
InferenceRule = (id, metavariables, premise_schemas, conclusion_schema,
                 type_constraints, side_conditions, freshness,
                 assumption_introduction, discharge_policy, provenance, hash)
```

```
verified_step(r, premises, subst, context) =
    rule_resolved and premises_available and types_match
    and substitution_capture_free and side_conditions_pass
    and freshness_pass and scope/discharge_valid
    and dependencies_acyclic_or_profile_valid
```

Rule family	Example contract	Typical rejection
Introduction	construct a compound proposition from checked subproofs	missing branch or undischarged assumption
Elimination	extract/use information licensed by the major premise	wrong connective/type or unavailable premise
Equality/rewrite	replace equals under a declared congruence/context	orientation, match, scope or equality-profile failure
Quantifier/binder	instantiate/generalize with type and freshness conditions	variable capture or eigenvariable violation
Computational reflection	decision procedure emits a proof/certificate for a finite proposition	unsupported operator or unverifiable oracle result
Induction/coinduction	well-founded or guarded cyclic obligations under a profile	missing measure, case or guardedness certificate

NO SYMBOL-ONLY RULE

A glyph such as →, =, forall or turnstile does not select a logic, precedence, introduction/elimination rules or semantic consequence. The exact OperatorCell and TheoryProfile do.

Proof DAG, Obligations and Deterministic Commit

State	Meaning
candidate	suggested node not yet kernel-checked
verified-local	node rule and immediate dependencies check, but root may still have open obligations
open	goal or side condition remains unresolved
closed	all dependencies included and kernel-replayable under the declared profile
committed	closed root atomically added to theorem registry with lineage
retracted/superseded	registry history records invalidation or a stronger/new version; hashes remain replayable

```
candidate proof graph -> resolve dependency closure -> topological order
 -> kernel replay every node -> require zero open obligations
 -> issue ProofCertificate -> atomic theorem-registry patch
 -> append pre/post registry hashes and proof lineage
```

CYCLE POLICY

Ordinary proof mode requires a finite DAG. Cyclic, recursive or coinductive proofs are rejected unless a dedicated guarded/fixed-point profile defines their acceptance and verifier.

Versioning and retraction

A theorem statement can retain stable identity while acquiring a new proof, stronger assumptions, a migrated theory profile or a retraction event. The lineage records what changed; hashes do not erase historical dependency.

Kernel, Equality and Rewrite Discipline

Relation	Meaning	Must not be confused with
definitional equality	terms normalize/convert under the theory profile	a user-proved equation or numeric approximation
propositional equality	a theorem/object in the logic	host-language object identity
approximate equality	tolerance/interval relation with certificate	exact equality
semantic equivalence	same interpretation under a named model/profile	glyph homeomorphism or string equality
content identity	same canonical record hash	truth, proof, authorship or semantic equivalence

```
rewrite trace = (theorem_id, direction, match_position, substitution,
                 context/congruence, pre_term_hash, post_term_hash, status)
```

SMALL-KERNEL PRINCIPLE

Complex automation is allowed outside the trusted core when it emits a compact proof object or certificate that a small independent checker can replay. Unsupported oracle output remains an unproved suggestion.

Semantics, Countermodels and External Oracles

Adapter	May establish	Must report
finite model checker	truth/counterexample in a named finite transition/model profile	model, bounds, state identity and certificate
SAT/SMT solver	satisfiable/unsatisfiable under encoded theory fragment	encoding, assumptions, proof/core/countermodel and unsupported features
computer algebra	exact identity or bounded result in supported domain	domain, side conditions, normalization and certificate/status
external theorem prover	proof object in imported logic	logic mapping, trust boundary and independent verification
numeric oracle	interval/residual for a proposition involving computation	precision, rounding, error and non-proof status where applicable
evidence ledger	empirical premises with provenance/uncertainty	observation is not derived by pure logic

```
syntactic derivability:  Gamma |- P
semantic consequence:      Gamma |=_M P
The two relations remain distinct until an explicit soundness/completeness theorem connects them for
the selected profile.
```

UNKNOWN IS VALID

Budget exhaustion, unsupported operators, missing certificates, open obligations and ambiguous side conditions return UNKNOWN or OPEN. They never silently become false, true or proved.

Unicode Notation and OperatorCells

Literal / fallback	Typed operator role	Boundary
forall / ∀	binder with domain, variable and body ports	glyph does not define logic or variable scope
exists / ∃	existential binder and witness/elimination rules	search success is not proof unless witness/rules check
and / ∧, or / ∨	connective under a theory profile	truth table/introduction rules are profile data
implies / → / ⇒	implication connective or function/transition arrow by exact OperatorCell	overloaded arrows must resolve before parsing
turnstile / |- / ⊢	syntactic derivability judgment	not semantic consequence or physical support
models / |= / ⊨	semantic satisfaction/consequence	requires an interpretation/model profile
equals / =	typed equality relation	not visual similarity or approximate equality
member / ∈	typed membership relation	Foundation converse and glyph geometry remain distinct faces

THREE FACES, ONE ADDRESS

Foundation 5.0 allows literal spelling, canonical glyph and typed semantic operator to share a content-addressed cell [S1]. PROOFWEAVE uses that atlas while keeping typography, syntax and proof truth separate.

Deterministic Proof Demonstration

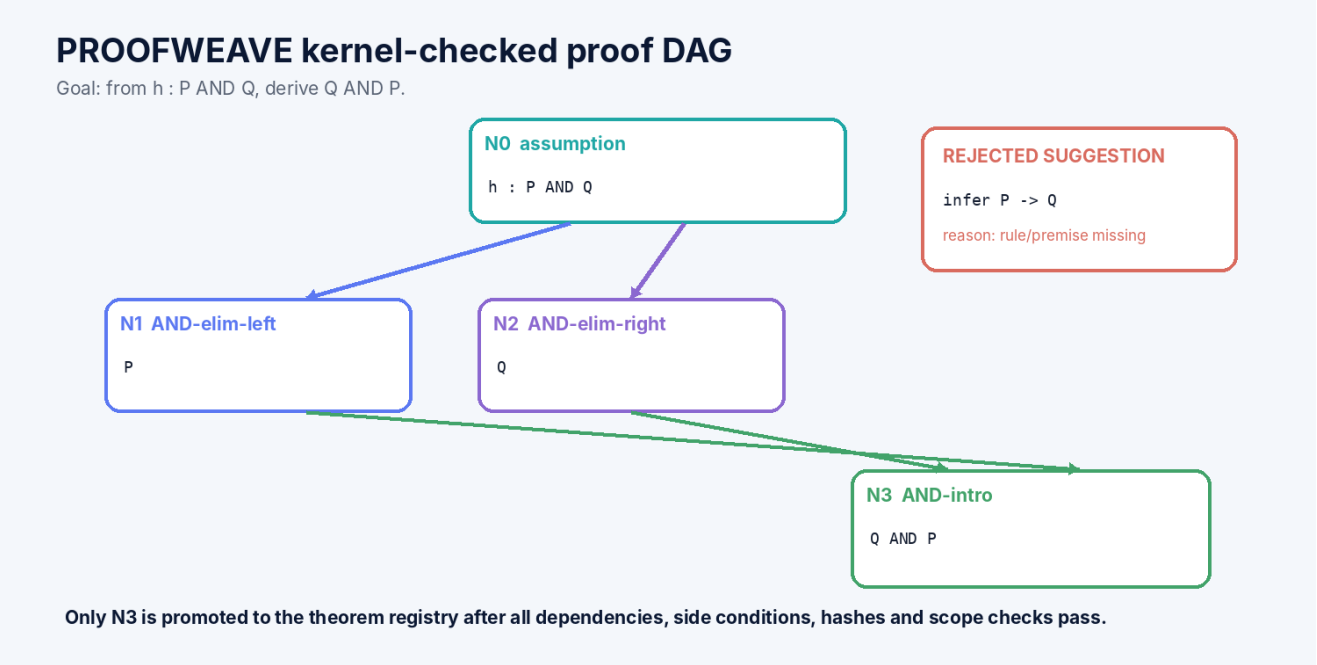


Figure 3. A four-node proof closes only after every rule application and dependency checks.

```
theory: propositional conjunction
context: h : P AND Q
N1 = and_elim_left(h)  : P
N2 = and_elim_right(h) : Q
N3 = and_intro(N2,N1)  : Q AND P
root = N3; open obligations = 0 -> certificate PASS
```

Candidate	Result	Reason
and_elim_left(h)	ACCEPT	premise type P AND Q matches; result P
and_elim_right(h)	ACCEPT	premise type P AND Q matches; result Q
and_intro(N2,N1)	ACCEPT	both premises available in same context
infer P → Q without rule/premise	REJECT	rule-or-premise-missing
reference N3 from its own dependency chain	REJECT	proof-cycle
commit root while side obligation remains	REJECT	open-obligations

Proof queries

- check_step(node_id) → pass/fail, substitutions, side conditions and exact reason
- open_obligations(root_id) → ordered contexts/goals and dependency paths
- explain(root_id, profile) → child/plain/formal projection from the same proof object
- trust_report(root_id) → axioms, admitted facts, external certificates and kernel version
- replay_certificate(bytes) → independently reconstructed root hash and theorem status

Certificate Format, Packing and Adapters

Certificate field	Purpose
theory/profile hash	fixes logic, types, definitions, axioms, kernel and normalization
statement/context	exact theorem root and assumptions
node table	rule IDs, premises, substitutions, side conditions, scope and result hashes
dependency commitments	external theorems/definitions and exact versions
trust/capability	kernel-checked, externally certified, bounded computation, admitted or unknown
integrity/framing	canonical serialization, codebook version, CRC/hash and migration profile
lineage	proof version, authoring/import events, retractions and registry pre/post hashes

PACKING RULE

Compact rule IDs, substitutions and shared DAG nodes may reduce storage only after the exact cold theory and codebook resolve. Unknown opcodes reject; a seed cannot reconstruct external theorem imports or human choices that were not stored.

Projection/adapter	Boundary
visual proof graph	layout and color are presentation; dependency IDs and rule records remain authority
natural-language explanation	may summarize a checked proof but cannot replace omitted kernel steps
educational/tactile view	preserves object identity, premises, rule and conclusion through another channel
tactic/AI assistant	proposal-only unless it emits a kernel-checkable object
external prover/model finder	versioned logic mapping and independently checkable certificate required

Reference Validation, Boundaries and Kill Criteria

REFERENCE SELF-CHECK RESULT

28/28 deterministic specification self-checks passed. These checks cover catalog uniqueness, required fields, dependency closure, demo arithmetic and declared reason codes. They are package evidence, not independent domain validation.

Checked area	Result
Catalog integrity	48 unique PFW IDs; required fields present.
Proof fixture	P AND Q |- Q AND P closes with four nodes and zero open obligations.
Kernel rejection	Missing rule, proof cycle and open-obligation fixtures return stable reasons.
Content addressing	Canonical node order and hashes are deterministic under alpha-normalization profile.
Trust boundary	External suggestion cannot enter theorem registry without a supported certificate/verifier.
Exactness boundary	Budget-exhausted search remains UNKNOWN and cannot emit a theorem certificate.

Explicit non-claims

- No universal logic, foundation of mathematics, consistency proof or completeness result is claimed.
- No theorem beyond the bundled finite fixture is established by this PDF itself.
- No AI, tactic or external solver output is trusted without a supported replayable certificate.
- No empirical premise is validated by pure proof structure alone.
- No proof-assistant, SMT, CAS or theorem-prover compatibility is claimed until adapters and mappings are tested.

Kill criteria

1. Reject or narrow a use when a symbol, codebook slot or packed record cannot resolve to its exact global definition.
2. Reject when a capability or exactness label is stronger than the model, measurement or external source supports.
3. Reject when units, profiles, coordinate systems, operand order or dependency order are implicit.
4. Reject when a proposal bypasses support, compatibility, guards, deterministic commit or lineage.
5. Reject when an external adapter is required for safety or conformance but its version, calibration or provenance is absent.
6. Reject any performance, compression or safety claim without a named workload, target, error/equivalence contract and evidence.

Complete PROOFWEAVE 1.0 Mechanism Catalog

The catalog contains 48 cartridge-local mechanisms. These IDs are component-scoped and do not silently extend a global M-number ladder.

ID	Family	Mechanism	Core contract
PFW001	Theory	Theory profile	Declares logic, sorts, equality, rule set, trust base, classical/constructive choices and version.
PFW002	Theory	Sort or type	Stable sort/type definition with formation, equality and coercion rules.
PFW003	Theory	Symbol declaration	Typed constant, function, relation or binder with arity and scope.
PFW004	Theory	Definition record	Content-addressed definiens/definiendum, dependencies, transparency and unfolding policy.
PFW005	Theory	Axiom or axiom schema	Explicit trusted proposition/schema; provenance and applicability are never hidden.
PFW006	Theory	Context	Ordered or set-like assumptions, local declarations and discharge state under a profile.
PFW007	Syntax	Term AST	Canonical typed term tree with binder and source-spelling references.
PFW008	Syntax	Proposition AST	Canonical formula tree distinct from its rendered notation.
PFW009	Syntax	Free and bound variable record	Binder identity and scope prevent accidental capture.
PFW010	Syntax	Substitution	Typed capture-avoiding replacement with domain, codomain and trace.
PFW011	Syntax	Alpha normalization	Canonical renaming profile for binder-insensitive identity and hashing.
PFW012	Syntax	Expression content hash	Canonical syntax/definition identity; hash is not truth or authorship proof.
PFW013	Inference	Inference rule	Premise schemas, conclusion schema, metavariables, side conditions and discharge policy.
PFW014	Inference	Premise matcher	Matches available proof nodes to rule premises with typed substitutions.
PFW015	Inference	Conclusion constructor	Builds the candidate conclusion after successful matching and normalization.
PFW016	Inference	Side condition	Decidable or externally certified predicate required by a rule application.
PFW017	Inference	Freshness condition	Checks variable, name or eigenvariable freshness against exact scope.
PFW018	Inference	Scope and discharge	Introduces, tracks and discharges assumptions through explicit rule semantics.
PFW019	Proof graph	Proof node	Conclusion, rule, premises, substitutions, scope, certificate and content hash.
PFW020	Proof graph	Dependency edge	Directed proof dependence distinct from visual layout or reading order.
PFW021	Proof graph	Open obligation	Unproved goal with context, constraints, priority and status UNKNOWN.
PFW022	Proof graph	Goal stack or agenda	Search/editor projection of obligations; not part of theorem truth by itself.
PFW023	Proof graph	Acyclic proof DAG	Closed finite derivations reject dependency cycles unless a cyclic profile is declared.
PFW024	Proof graph	Cyclic/fixed-point proof profile	Guarded coinduction or fixed-point semantics require a dedicated soundness contract.

ID	Family	Mechanism	Core contract
PFW025	Kernel	Type and sort check	Every term, premise and conclusion is checked against the declared theory.
PFW026	Kernel	Definitional equality	Profile-bound normalization or conversion, separate from asserted propositional equality.
PFW027	Kernel	Rewrite step	Uses an oriented theorem/definition with match, scope and termination/status trace.
PFW028	Kernel	Congruence step	Lifts checked equality/relation through a declared context or operator.
PFW029	Kernel	Normalization certificate	Records normal form, strategy, fuel and residual/open status.
PFW030	Kernel	Kernel proof check	Replays every node from the small trusted rule base before theorem commit.
PFW031	Semantics	Model interpretation	Maps symbols into a declared structure; interpretation is distinct from syntax.
PFW032	Semantics	Entailment profile	Defines semantic consequence scope and finite/decidable checking boundary.
PFW033	Semantics	Countermodel adapter	External model finder may refute a candidate under a named finite/profile scope.
PFW034	Oracle	Decision-procedure adapter	SAT/SMT/algebra/rewriting oracle returns a scoped certificate or UNKNOWN.
PFW035	Oracle	External prover certificate	Imports proof objects only through a versioned independent verifier.
PFW036	Oracle	Trust boundary	Declares kernel, axioms, admitted facts, external certificates and unchecked assumptions.
PFW037	Search	Proof suggestion	Human, tactic, search or AI proposal with no theorem authority.
PFW038	Search	Tactic trace	Expands one goal into candidate subgoals with deterministic parameters and budget.
PFW039	Search	Budget and UNKNOWN	Time, nodes, depth and solver limits terminate without inventing a proof.
PFW040	Authority	Verified proof commit	Promotes a closed kernel-checked proof to theorem registry atomically.
PFW041	Authority	Theorem registry patch	Adds theorem ID, statement, proof root, dependencies, profile and hashes.
PFW042	Authority	Proof lineage	Records proof versions, rewrites, imported facts, migrations, retractions and replay.
PFW043	Notation	Unicode OperatorCell	Literal symbol, glyph, typed logical operator and ports remain co-addressed but distinct.
PFW044	Notation	Direct and converse relation spelling	Surface operand order maps to canonical typed ports without changing truth value.
PFW045	Serialization	Proof certificate format	Canonical theorem, context, nodes, dependencies, exactness and trust metadata.
PFW046	Validation	Independent verifier	Small separate checker rejects missing nodes, bad hashes, scope errors and unsupported rules.
PFW047	Projection	Educational and visual proof view	Natural-language, tactile or graph projections preserve the same underlying proof object.
PFW048	Governance	Proof cartridge conformance report	Checks theory closure, kernel fixtures, certificates, trust labels, traces and explicit non-claims.

Claims Ledger

Claim	Disposition	Reason
A closed kernel-checked derivation proves its conclusion in the declared theory.	ADMIT / profile	Subject to the exact kernel, axioms, definitions and rule profile named by the certificate.
A content hash proves truth.	REJECT	It proves record identity/integrity, not soundness or truth.
A pretty proof graph is a proof.	REJECT	Every node, dependency, scope and side condition must replay in the kernel.
A theorem in one theory is universally true.	REJECT	The result is scoped to assumptions, logic, definitions and semantics.
Syntactic derivability and semantic entailment are identical by notation.	REJECT	Turnstile and semantic consequence are distinct typed relations unless a metatheorem connects them.
AI or tactic output may commit a theorem directly.	REJECT	Suggestion remains proposal-only until kernel verification.
Failure to find a proof refutes the goal.	REJECT	Budget exhaustion returns UNKNOWN unless a checked countermodel or refutation is supplied.
Normalization always terminates.	REJECT	Termination depends on the rewrite/profile; fuel and residual status remain explicit.
Cyclic proofs are always invalid.	CORRECT / profile	Ordinary finite DAG mode rejects cycles; guarded cyclic/coinductive proofs need a separate soundness contract.
A local proof proves theory consistency.	REJECT	Consistency is a separate metatheoretic claim.
A proof of a scientific statement proves the observations.	REJECT	Empirical premises require evidence and provenance outside the pure proof kernel.
PROOFWEAVE is a universal automated mathematician.	REJECT	It is a proof-carrying substrate and verifier architecture, not a claim of complete automation.

High-Impact Application Profiles

Profile	What the cartridge enables
Proof-carrying game solver layer	Give RULEFORGE, Chess and Go exact certificates a common proof DAG, trust and replay envelope.
Executable mathematical notebook	Separate definitions, calculations, bounded numerics and actual theorems inside one inspectable record.
Scientific derivation ledger	Connect evidence-bearing premises to formal transformations without claiming that proof structure creates observations.
Configuration and protocol verification	Carry invariants, transition proofs, counterexamples and exact dependency lineage.
Child-to-expert proof lab	Render the same proof object as cards, tactile links, plain-language steps or full formal notation.

Final Formal Definition

FORMAL DEFINITION

UGTOMS PROOFWEAVE 1.0 is a component-scoped proof cartridge in which theories, sorts, symbols, content-addressed definitions, contexts, inference rules, proof nodes, obligations, semantic/oracle interfaces, trust labels and theorem lineage are typed substrate data; search and explanation remain proposal/projection layers; and only a closed independently replayable kernel certificate may atomically promote a statement into the theorem registry.

Implementation sequence

7. Freeze schemas for TheoryProfile, InferenceRule, ProofNode, ProofCertificate and TheoremRegistry.
8. Implement a tiny propositional/equality kernel, canonical AST hashing and independent verifier.
9. Ship the conjunction proof, invalid-rule, cycle and open-obligation fixtures.
10. Add versioned SAT/SMT and exact-computation certificate adapters only where independently checkable.
11. Integrate proof certificates into RULEFORGE, Chess, Go, EVIDENCE and later domain cartridges.

RELEASE CONCLUSION

This 1.0 document freezes a domain-facing contract. It is deliberately useful before a production engine exists: implementations can now disagree visibly through typed profiles, reason codes and certificates instead of hiding differences behind the same symbol.